Facultatea de Automatică şi Calculatoare, UPB
Departamentul de Calculatoare

Analiza algoritmilor

ŞL.dr.ing. Andrei Mogoş

2020-2021

- Corectitudinea algoritmilor (1) -

Corectitudinea algoritmilor

Considerăm un algoritm Alg cu date de tip $\underline{I}$ și rezultate de tip O.

- Datele $d \in \underline{I}$ sunt considerate <u>corecte</u> dacă satisfac un predicat $P_{\underline{I}} : \underline{I} \to \{0,1\}$ numit <u>asertiune de intrare</u>.

• Rezultatele algoritmului pentru datele $d \in \underline{I}$, rezultate notate cu $r = Alg(d)$, $r \in O$, sunt considerate <u>corecte</u> dacă satisfac un predicat $P_O : I \times O \to \{0,1\}$ numit <u>asertiune de ieșire</u>.

<u>Obs</u>: Cele două asertiuni caracterizează problema rezolvată.

Ex 1 (P1): Să se calculeze factorialul unui număr natural nenul.

```
fact(n)
|  if n = 1
|  |  return 1
|  else
|  |  return n * fact(n - 1)
```

$I = \mathbb{N}$

$O = \mathbb{N}$

$P_I(n) =_{def} n \geq 1$

$P_O(n, r) =_{def} r = \prod_{k=1}^{n} k$

Def: Fie Alg un algoritm cu date de tip I și cu rezultate de tip O.

a) Algoritmul Alg s.n. **parțial corect** în raport cu asertiunile P_I și P_O dacă:

$\forall d \in I$ cu proprietatea că

$P_I(d) = 1$ și se_termină(Alg, d)

avem $P_O(d, Alg(d)) = 1$.

b) Algoritmul Alg s.n. total corect în raport cu asertiunile P_I și P_O dacă:

$\forall d \in I$ cu proprietatea că

$$P_I(d) = 1$$

avem se_termină(Alg, d) și $P_O(d, Alg(d)) = 1$.

Ex 2: Fie algoritmul:

```
plus(x, y)
    if x = 0
        return y
    else
        return 1 + plus(x - 1, y)
```

(P2): Să se calculeze suma a două numere întregi.

$I = Z' \times Z'$

$O = Z'$

$P_I(x, y) =_{def} 1$

$P_O((x, y), r) =_{def} r = x + y$

• Se demonstrează prin inducție după x că pentru orice intrare corectă ($P_I(x, y) = 1$) pentru care algoritmul plus se termină avem relația:

$$plus(x, y) = x + y.$$

$\Rightarrow$ algoritmul plus este <u>parțial corect</u> în raport cu aserțiunile P_I și P_O.

- Se observă că algoritmul plus se termină doar pentru intrări corecte ($P_I(x, y) = 1$) din mulțimea $\mathbb{N} \times \mathbb{Z}$

(de exemplu, pentru perechea $(-1, 5)$ algoritmul plus nu se termină (calculul plus$(-1, 5)$ nu se termină)).

$\Rightarrow$ algoritmul plus <u>nu</u> este <u>total</u> <u>corect</u> în raport cu asertiunile P_I și P_O.

Ex 3 : Fie algoritmul:

```
plus (x, y)
|   if x = 0
|   |   return y
|   else
|   |   return 1 + plus (x-1, y)
```

(P3): Să se calculeze suma a două numere naturale.

$I = \mathbb{N} \times \mathbb{N}$

$O = \mathbb{N}$

$P_{\underline{I}}(x, y) =_{def} 1$

$P_O((x, y), r) =_{def} r = x + y$

- Analog ca problema (P2) se arată că algoritmul plus este <u>parțial corect</u> în raport cu asertiunile $P_{\underline{I}}$ și P_O.
- În plus, algoritmul plus se termină pentru orice intrare corectă ($P_{\underline{I}}(x, y) = 1$)

$\Rightarrow$ algoritmul plus este <u>total corect</u> în raport cu asertiunile P_I și P_O.

<u>Obs</u>: Pentru a analiza corectitudinea parțială și corectitudinea totală a unui algoritm trebuie să se ia în considerare atât algoritmul respectiv, cât și problema care trebuie rezolvată.

Oss:

Analiza corectitudinii unui algoritm nu are sens fără existența unei asertiuni de intrare și a unei asertiuni de ieșire care să descrie problema.

Obs:

1) Prin verificarea corectitudinii parțiale vom înțelege să demonstrăm că asertiunea de ieșire este adevărată pentru orice date care satisfac asertiunea de intrare.

2) Terminarea unui algoritm o vom considera fie evidentă, fie o vom explica intuitiv.

Metode pentru verificarea corectitudinii parțiale

1. Deducție:
   - directă
   - prin reducere la absurd
2. Inducție bine formată
   - inducție completă
   - inducție matematică
   - inducție structurală

1. Deducția

Deducția directă

ipoteză ... => ... concluzie

Deducția prin reducere la absurd

not (concluzie) ... => ... not (ipoteză)

Ex (pentru deducție directă)

Fie n un număr natural. Definim rădăcina pătrată întreagă a lui n ca fiind numărul natural $isqr(n)$ cu proprietatea:

$$(isqr(n))^2 \leq n < (isqr(n)+1)^2$$

$isqr(0) = 0 : 0 \leq 0 < 1$
$isqr(1) = 1 : 1 \leq 1 < 4$
$isqr(2) = 1 : 1 \leq 2 < 4$

$isqr(3) = 1 : 1 \leq 3 < 4$
$isqr(7) = 2 : 4 \leq 7 < 9$

Să se construiască un algoritm care să calculeze $isqr(n)$.

$I = \mathbb{N}$
$O = \mathbb{N}$

$P_I(n) =_{def} 1$
$P_O(n, r) =_{def} r = isqr(n)$

$\prod_O^I$ Construim algoritmul demonstrând o formulă de calcul pentru $isqr(n)$.

Considerăm $n > 0$. Fie $q = \lfloor n/4 \rfloor$. Notăm $k = isqr(q)$.

Rezultă că: $k^2 \leq q < (k+1)^2$ (1)

$q < (k+1)^2$
$q, (k+1)^2 \in \mathbb{N}$ $\Big| \Rightarrow q+1 \leq (k+1)^2 \Rightarrow$

$\Rightarrow 4q+4 \leq (2k+2)^2$ (2)

Avem inegalitățile:

$$\lfloor n/4 \rfloor \leq n/4 < \lfloor n/4 \rfloor + 1$$

$$(\lfloor a \rfloor \leq a < \lfloor a \rfloor + 1)$$

$\Rightarrow q \leq n/4 < q+1$

$\Rightarrow \underline{4q \leq n < 4q+4}$

Din (1): $k^2 \leq q < (k+1)^2 \Rightarrow (2k)^2 \leq 4q$

Din (2): $4q+4 \leq (2k+2)^2$

Rezultă că:

$(2k)^2 \leq 4q \leq n < 4q+4 \leq (2k+2)^2$

$\Rightarrow \boxed{(2k)^2 \leq n < (2k+2)^2}$

$$(2k)^2 \le n < (2k+2)^2$$

caz 1 : $(2k)^2 \le n < (2k+1)^2$

$\Rightarrow isqr(n) = 2k \Rightarrow$

$\Rightarrow \boxed{isqr(n) = 2\, isqr(\lfloor n/4 \rfloor)}$

caz 2 : $(2k+1)^2 \le n < (2k+2)^2$

$\Rightarrow isqr(n) = 2k+1 \Rightarrow$

$\Rightarrow \boxed{isqr(n) = 2\, isqr(\lfloor n/4 \rfloor) + 1}$

```
Alg_isqr(n)
    if n = 0
        return 0
    else
        k = Alg_isqr(⌊n/4⌋)
        if n < (2k+1)^2
            return 2k
        else
            return 2k+1
```

- Alg – isqr este <u>parțial corect</u> în raport cu asertiunile P_I și P_O. Acest lucru rezultă din demonstrația în urma căreia am aflat formula de calcul pentru isqr(n). (*)
- Se observă că Alg – isqr se termină pentru orice intrare corectă. (**)

Din (*) și (**) rezultă că Alg – isqr este <u>total corect</u> în raport cu asertiunile P_I și P_O.

Analiza complexității algoritmului Alg-isqr

$$T(n) = \begin{cases} K_1, & n = 1 \\ T(\lfloor n/4 \rfloor) + K_2, & n > 1 \end{cases}$$

Aplicăm metoda Master:

$$T(n) = T(n/4) + \Theta(1)$$

$a = 1$, $b = 4$ $\Rightarrow$ $n^{\log_b a} = n^{\log_4 1} = n^0 = 1$

$f(n) = \Theta(1) = \Theta(n^{\log_b a})$ (pt dimensiunea problemei n)

caz 2 $\Rightarrow$ $T(n) = \Theta(n^{\log_b a} \cdot \lg n) = \Theta(\lg n)$

Dacă notăm h = numărul de biți din reprezentarea lui n $\Rightarrow$ dimensiunea problemei este $h \simeq \lg n$

$\Rightarrow T(h) = \Theta(h)$

În concluzie, în ambele cazuri (și pentru n și pentru h) complexitatea algoritmului este foarte bună $\Rightarrow$

$\Rightarrow$ un algoritm rapid.